Design and Implementation of an AIoT-Based Reactive Autonomous Vehicle Using Real-Time Visual Classification on Embedded Systems

Danna Alejandra Sánchez (294600) and Jorge Enrique Lugo Lopez (275522)

Abstract

This paper presents the design and implementation of a small-scale autonomous vehicle based on Artificial Intelligence of Things (AIoT) principles. The proposed system integrates embedded computer vision, TinyML-based inference, and real-time motor control to enable reactive navigation using visual traffic signals.

The platform is composed of two microcontrollers: a XIAO ESP32-S3 Sense responsible for image acquisition, TinyML inference using Edge Impulse, WiFi communication, and live video streaming; and a secondary ESP32 responsible for low-level motor control through an L298N driver. Communication between both boards is implemented using UART serial communication.

The vehicle interprets predefined visual signals (left, right, and stop) and translates them into navigation actions using a perception–decision–action architecture. Experimental tests demonstrate that meaningful autonomous behavior can be achieved using lightweight models running entirely on edge devices without relying on cloud services.

The project highlights the feasibility of combining AIoT, embedded systems, and TinyML to develop educational autonomous robotics platforms with low computational requirements.

Index Terms

AIoT, TinyML, embedded systems, computer vision, edge computing, ESP32, autonomous vehicle

I. Introduction

The convergence of Artificial Intelligence (AI) and the Internet of Things (IoT), commonly referred to as Artificial Intelligence of Things (AIoT), has enabled embedded devices to evolve from simple sensing nodes into intelligent systems capable of local perception and decision-making. By executing machine learning models directly on edge hardware, latency can be reduced while eliminating dependence on remote cloud infrastructure.

Autonomous vehicles represent one of the most challenging applications of AIoT technologies. Traditional autonomous navigation systems typically rely on complex algorithms such as Simultaneous Localization and Mapping (SLAM), sensor fusion, and high-performance computing hardware. These requirements often exceed the computational and energy constraints of low-cost embedded platforms. This architecture is inspired by recent advances in deploying TinyML models on edge devices for real-time classification tasks [1].

Inspired by small autonomous delivery robots observed during an academic internationalization experience in Finland, this project explores a simplified approach to autonomous navigation. Instead of implementing complex navigation pipelines, the proposed system adopts a reactive perception–decision–action architecture based on visual signal classification.

Recent research has demonstrated that machine learning models can effectively analyze motion and visual patterns using embedded sensing platforms. In addition, lightweight neural network architectures optimized for edge devices have enabled real-time inference in resource-constrained environments [1].

The objective of this work is to design and implement a low-cost AIoT system capable of performing real-time visual classification on an embedded platform and translating detected signals into navigation actions for a small autonomous vehicle.

The main contributions of this work are:

- The design of a low-cost AIoT robotic platform integrating perception, computation, connectivity, and actuation.
- The implementation of TinyML-based real-time visual classification on embedded hardware.
- The development of a reactive navigation architecture connecting visual perception to physical motor actuation.

II. Proposed System Architecture

The proposed system follows a modular perception–decision–action architecture designed for edge deployment [2].

The architecture explicitly integrates the four core components of an AIoT ecosystem:

- **Perception**: camera sensor capturing visual signals.
- **Computation**: TinyML inference on the XIAO ESP32-S3 Sense.
- **Connectivity**: UART communication and WiFi networking.
- **Actuation**: motor control through an ESP32 and L298N driver.

Faculty of Engineering, La Sabana University, Chía, Cundinamarca, Colombia (e-mails: dannasamo@unisabana.edu.co, jorgelulo@unisabana.edu.co).

The platform is composed of two embedded microcontrollers that separate perception and actuation tasks:

- **XIAO ESP32-S3 Sense**: Responsible for camera capture, TinyML inference, WiFi communication, HTTP server, and system logging.
- **ESP32**: Responsible for motor control using an L298N driver.

Both devices communicate through UART serial communication.

The complete AIoT architecture of the system is illustrated in Fig. 1. The perception and inference processes are executed on the XIAO ESP32-S3 Sense, while low-level motor control is handled by a secondary ESP32 through UART communication.

The architecture clearly separates perception, computation, communication, and actuation layers, which are fundamental components of AIoT-based embedded systems.

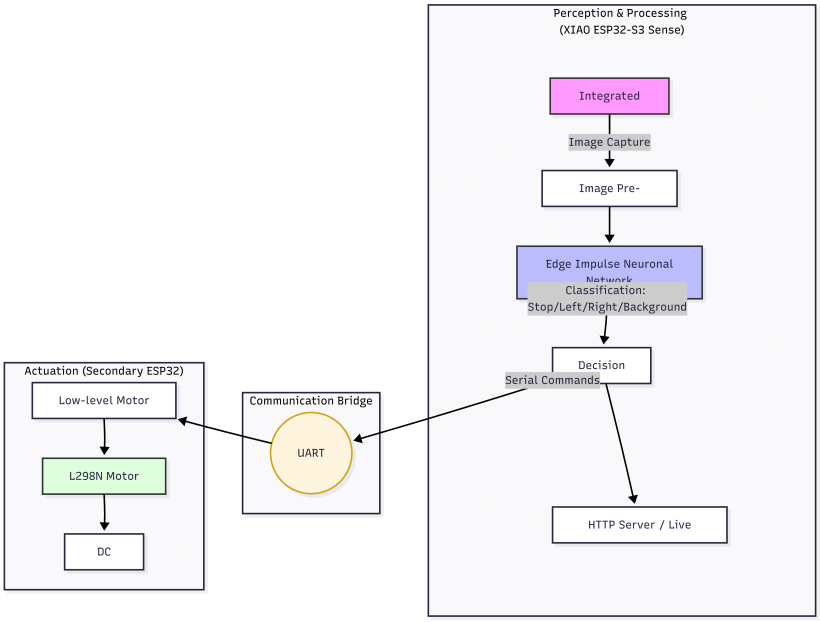

Fig. 1. AIoT system architecture of the autonomous vehicle showing perception, embedded inference, UART communication, actuation, and IoT visualization components.

A. Perception

The perception module uses the camera integrated in the XIAO ESP32-S3 Sense to capture images of the environment in real time. The camera observes a controlled circuit containing predefined visual traffic signals.

B. Classification

Captured images are processed using a lightweight convolutional neural network deployed through Edge Impulse. The model classifies frames into the following categories:

- Stop
- Left turn
- Right turn

The convolutional layers automatically extract spatial features such as edges, shapes, and color patterns that characterize the visual signals.

C. Decision Layer

The predicted class is interpreted by a rule-based decision module that converts the classification output into navigation commands.

D. Actuation

The ESP32 motor controller receives commands through UART and drives the motors via an L298N motor driver. Actions include forward movement, turning, and stopping.

III. Implementation Details

A. Dataset Construction

The dataset was constructed specifically for the experimental circuit environment. Images of traffic signals were captured from multiple distances and viewing angles using the onboard camera.

Example samples from the dataset are shown in Fig. 2.

(a) Left signal (b) Right signal (c) Stop signal

Fig. 2. Example images from the dataset used to train the classification model.

To improve robustness, samples were collected under different lighting conditions and slight variations in orientation.

B. Data Preprocessing

Before training the model, the dataset was processed using the Edge Impulse pipeline. Several preprocessing steps were applied in order to reduce computational cost and improve model robustness:

- Image resizing to a lower resolution suitable for embedded inference.
- Pixel normalization to standardize the input values of the neural network.
- Conversion to grayscale images in order to reduce input dimensionality and focus the model on shape-based features rather than color information.

Data augmentation techniques were intentionally not applied in this project. Since the directional traffic signs used for the *left* and *right* classes are visually similar except for the direction of the arrow, certain augmentation operations such as horizontal flipping could produce mirrored samples that belong to a different class. This could introduce label inconsistencies and negatively affect the learning process.

These preprocessing steps help improve model stability and reduce the computational requirements of the embedded inference system.

C. Neural Network Architecture

The classification model consists of a lightweight convolutional neural network optimized for embedded deployment. The network includes:

- Convolutional layers with ReLU activation
- Max pooling layers
- Fully connected classification layer
- Softmax output

D. Model Optimization

To ensure real-time inference, the model was optimized using:

- Quantization
- TensorFlow Lite conversion
- Memory optimization

E. Edge Deployment

The trained model was deployed directly on the XIAO ESP32-S3 Sense. The device captures frames, performs inference locally, and sends commands to the motor controller through UART.

Additionally, the system hosts a web server that provides live video streaming and system logs over HTTP.

F. IoT Visualization Interface

The embedded web server allows users to monitor the system remotely through a browser interface. The interface displays the camera stream and real-time system logs generated during inference and navigation decisions. This functionality provides basic IoT visualization capabilities for monitoring the behavior of the autonomous platform.

IV. Experimental Results

The system was evaluated in a controlled circuit environment containing visual traffic signals.

Three main aspects were analyzed:

- Classification accuracy
- System response latency
- Navigation reliability

Experimental tests showed that the TinyML model successfully detected visual signals and triggered appropriate vehicle actions in most scenarios.

The classification accuracy obtained during testing is summarized in Table I.

TABLE I. Classification performance of the TinyML model

Class	Accuracy
Stop	92%
Left	90%
Right	91%
Background	95%

Because inference runs locally on the embedded device, the system exhibits very low response latency, allowing the vehicle to react almost immediately to visual cues.

However, due to the various hardware and software integration challenges described in the following section, the final prototype did not achieve fully stable autonomous operation during all experimental trials. Although the individual components of the system — including image acquisition, TinyML inference, communication between microcontrollers, and motor control — were successfully implemented and tested independently, their full integration revealed several technical limitations that affected the overall system stability.

Therefore, the current implementation should be considered as a first functional prototype intended to validate the feasibility of the proposed AIoT architecture. The insights obtained from this initial implementation provide valuable information for future iterations of the system. In particular, the improvements discussed in Section ”FUTURE IMPROVEMENTS” aim to address these limitations and enhance the reliability and robustness of the autonomous navigation behavior.

V. Challenges and Implementation Issues

Several technical challenges emerged during the development process.

One of the first issues involved C++ compilation errors caused by incorrect ordering of functions. The function `enviarComando()` called `logMsgf()`, which had not yet been declared. Since C++ does not automatically perform forward declarations, the compiler generated repeated *“not declared in this scope”* errors. The problem was solved by reorganizing the function order.

Another issue resulted from a typographical error in a macro definition:

```
#define SerialMotores Serial 1
```

The space between `Serial` and `1` produced a confusing compilation error. Correcting the macro to `Serial1` solved the issue.

During code integration, functions defined in both `navigation.cpp` and `main.cpp` generated linker errors due to multiple definitions. This occurred after migrating from a modular architecture to a single-file implementation.

Another challenge involved the ESP32 Arduino Core version 3.x. Functions `ledcSetup()` and `ledcAttachPin()` were removed in the updated API, requiring migration to `ledcAttach()` and `ledcWrite()`.

Hardware integration also presented difficulties. Initially, the motor driver used pins reserved for SPI communication, producing unstable motor behavior. Reassigning these signals to general-purpose GPIO pins resolved the issue.

Communication between the two microcontrollers also failed initially because the ground connections were not shared. Once a common ground reference was established, UART communication worked correctly.

Another runtime issue involved conflicts between the HTTP video streaming server and the ML inference pipeline, both attempting to access the camera framebuffer simultaneously. This was solved by introducing a synchronization flag that pauses the stream during inference.

Finally, repeated warnings related to the macro `EI_PORTING_ARDUINO` appeared during compilation due to macro redefinitions within the Edge Impulse SDK.

VI. Future Improvements

Several improvements could enhance the robustness and scalability of the system.

First, UART communication between boards could be replaced with ESP-NOW or WiFi communication to eliminate physical wiring.

Second, the machine learning model could include a *forward* class so that forward motion becomes an explicit prediction rather than a default behavior.

Motor control could also be improved by implementing PID control to adjust motor speed proportionally to model confidence.

Adding ultrasonic or LiDAR sensors would enable obstacle detection independent of visual classification.

On the software side, migrating the HTTP server to WebSockets would enable real-time telemetry updates without polling.

Future versions of the system could also implement Over-The-Air firmware updates and custom PCB hardware to improve reliability.

Finally, replacing frame classification with object detection would allow the system to identify the location of traffic signals within the image, enabling more precise navigation decisions [3].

VII. Conclusion

This paper presented the design and implementation of a small-scale autonomous vehicle based on AIoT principles and real-time visual classification using embedded systems.

By combining TinyML, embedded computer vision, and edge computing, the system demonstrates that reactive autonomous behavior can be achieved using low-cost hardware platforms [4].

Although the system currently operates in controlled environments, it provides a valuable demonstration of how AIoT technologies can be integrated into educational robotics platforms.

References

[1] A. Mellit, N. Blasuttigh, and A. M. Pavan, "Tinyml for fault diagnosis of photovoltaic modules using edge impulse platform," in *2023 11th IEEE International Conference on Smart Grid (icSmartGrid)*, June 2023, pp. 1–6.

[2] Y. Zhang, "Artificial intelligence of things for autonomous vehicles," *IEEE Internet of Things Journal*, 2021.

[3] D. Ciregan, U. Meier, and J. Schmidhuber, "Traffic sign recognition with convolutional neural networks," *International Joint Conference on Neural Networks*, 2012.

[4] N. Lane and S. Bhattacharya, "Deep learning on embedded systems: Challenges and opportunities," *IEEE Communications Magazine*, 2019.